ap-aging-sample · docs/ap_aging.md

Code documentation by the Backend agent · FEAT-ZD-48600

ap_aging module overview

`ap_aging` (Python 3.9, standard library only) works on open accounts-payable invoices. All functions live in `src/ap_aging/aging.py` and are pure: no I/O, no network, inputs are not modified. Money is `Decimal` throughout and never passes through `float`.

Invoice input

Each invoice is a plain `dict` (contract DC-1, as produced by `ap_pipeline`):

Key	Type	Notes
`id`	`str`	Invoice id
`vendor`	`str`	Vendor name
`due_date`	`datetime.date`	Due date
`amount`	`Decimal`	Open amount, 2 decimal places
`paid`	`bool`	Paid invoices are excluded everywhere

Extra keys (for example `paid_date`) are ignored.

Public functions (`ap_aging.__all__`)

`aging_buckets(invoices: Iterable[Dict], as_of: date) -> Dict[str, Decimal]`

Total open amount per bucket: `current`, `1-30`, `31-60`, `61-90`, `90+`. Unchanged.

`overdue_with_risk(invoices: Iterable[Dict], as_of: date, risk_by_vendor: Dict[str, str]) -> List[Dict]`

New in 1.5.0 (STORY-05). Lists the unpaid invoices that are **more than 30** calendar days past due as of `as_of`, each with its vendor's risk level.

- Inputs: the invoices; `as_of` (required, no default); `risk_by_vendor`, a map from vendor to `'high'`, `'medium'`

or `'low'` (contract DC-3, built from `ap_forecast.vendor_risk`).

- Rules:
- paid invoices are skipped (`inv.get("paid")`);
- `days = days_past_due(due_date, as_of)`; an invoice is kept only when `days > OVERDUE_AFTER_DAYS` (30), so an

invoice exactly 30 days past due is **not** listed. `bucket_for` is not used (`bucket_for(30)` returns `"31-60"`);

- `amount` is quantised to two decimals (`Decimal("980.5")` becomes `Decimal("980.50")`);
- a vendor missing from `risk_by_vendor` gets `'unknown'`.
- Output: a list of rows with exactly the keys `id`, `vendor`, `days` (`int`), `amount` (`Decimal`, 2 dp), `risk`,

sorted by `days` descending, then `id` ascending.

- Errors: an overdue unpaid invoice with no `vendor` key raises `KeyError`.

```
from datetime import date
from decimal import Decimal
from ap_aging import overdue_with_risk

invoices = [
    {"id": "SYN-INV-004", "vendor": "Synthetic Vendor A", "due_date": date(2026, 8, 11), "amount": Decimal("1250.00"), "
    {"id": "SYN-INV-005", "vendor": "Synthetic Vendor B", "due_date": date(2026, 9, 25), "amount": Decimal("15.00"), "pa
]
rows = overdue_with_risk(invoices, date(2026, 10, 25), {"Synthetic Vendor A": "high"})
# [{'id': 'SYN-INV-004', 'vendor': 'Synthetic Vendor A', 'days': 75, 'amount': Decimal('1250.00'), 'risk': 'high'}]
# SYN-INV-005 is exactly 30 days past due, so it is not listed.
```

`overdue_rows_to_json(rows: Iterable[Dict]) -> str`

New in 1.5.0 (STORY-06). Serialises the rows from `overdue_with_risk` to the CT-1 JSON contract read by `ap_portal`.

- Input: rows with `id`, `vendor`, `days`, `amount`, `risk`.
- Output: a JSON array of objects with keys in the order `id` (string), `vendor` (string), `days` (integer),

`amount` (two-decimal string, no exponent, no thousands separator), `risk` (string); rows keep their order; `'[]'` when there are no rows. There is no date field.

```
from ap_aging import overdue_rows_to_json

overdue_rows_to_json(rows)
# '[{"id": "SYN-INV-004", "vendor": "Synthetic Vendor A", "days": 75, "amount": "1250.00", "risk": "high"}]'
overdue_rows_to_json([])
# '[]'
```

The consumer parses the string and passes the array to `renderOverdueRisk` in `ap_portal`. No transport is added.

Helpers and constants (module level, not in `__all__`)

Name	Meaning
`BUCKETS`	The five bucket labels used by `aging_buckets`
`days_past_due(due_date, as_of) -> int`	Calendar days after the due date, 0 when not yet due
`bucket_for(days) -> str`	Bucket label for a day count (used by `aging_buckets` only)
`OVERDUE_AFTER_DAYS = 30`	Overdue threshold for `overdue_with_risk` (strict >)
`TWO_PLACES = Decimal("0.01")`	Quantum for two-decimal amounts